

DYNAMIC PROGRAMMING

- **Dynamic programming** is a technique that breaks the problems into sub-problems, and saves the result for future purposes so that we do not need to compute the result again.
- The subproblems are optimized to optimize the overall solution is known as optimal substructure property.
- The main use of dynamic programming is to solve optimization problems
- Here, optimization problems mean that when we are trying to find out the minimum or the maximum solution of a problem.
- The dynamic programming guarantees to find the optimal solution of a problem if the solution exists.

Consider an example of the **Fibonacci series**.

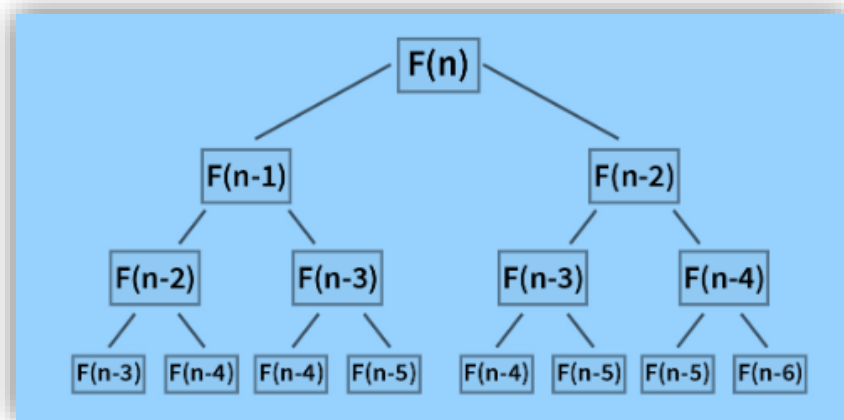
- The following series is the Fibonacci series: **0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ,...**
- The numbers in the above series are **not** randomly calculated. Mathematically, we could write each of the terms using the below formula:

$$F(n) = F(n-1) + F(n-2),$$

- With the base values **F(0) = 0**, and **F(1) = 1**. To calculate the other numbers, we follow the above relationship.

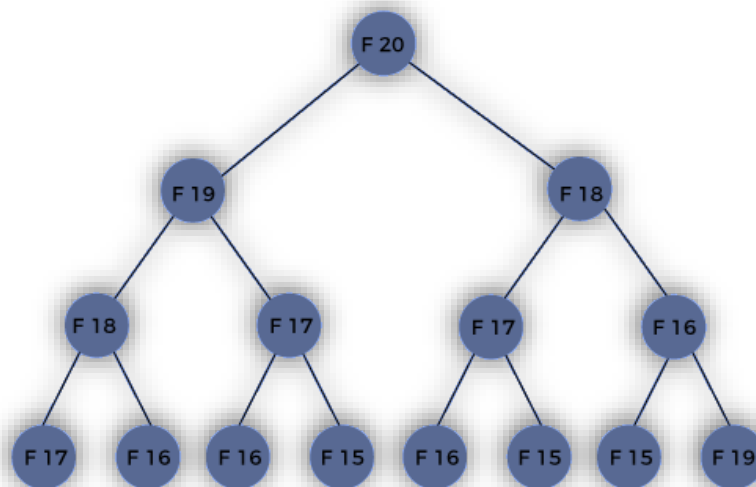
For example, **F(2)** is the **sum f(0) and f(1)**, which is equal to **1**

A recursion tree for the **Fibonacci** sequence:



How can we calculate $F(20)$?

The **F(20)** term will be calculated using the nth formula of the Fibonacci series. The below figure shows that how **F(20)** is calculated.



Approaches (method) of dynamic programming

There are **two** approaches to dynamic programming:

- **Top-down approach**
- **Bottom-up approach**

❖ **Top-down approach**

- ✓ **The top-down approach** follows the memorization technique, while bottom-up approach follows the tabulation method.
- ✓ Here memorization is equal to the sum of recursion and caching
- ✓ Recursion means calling the function itself, while caching means storing the intermediate results.

Advantages

- It is very easy to understand and implement.
- It solves the subproblems only when it is required.
- It is easy to debug.

Disadvantages

- It uses the recursion technique that occupies more memory in the call stack.
- Sometimes when the recursion is too deep, the stack overflow condition will occur.
- It occupies more memory that degrades the overall performance.

❖ **Bottom-Up approach**

- ✓ The bottom-up approach is also one of the techniques which can be used to implement the dynamic programming.
- ✓ It uses the tabulation technique to implement the dynamic programming approach.
- ✓ It solves the same kind of problems but it removes the recursion.
- ✓ If we remove the recursion, there is no stack overflow issue and no overhead of the recursive functions.
- ✓ In this tabulation technique, we solve the problems and store the results in a matrix.

There are **two ways** of applying dynamic programming:

- **Top-Down**
- **Bottom-Up**

- ✓ The bottom-up is the approach used to avoid the recursion, thus saving the memory space.
- ✓ The bottom-up is an algorithm that starts from the beginning, whereas the recursive algorithm starts from the end and works backward.
- ✓ In the bottom-up approach, we start from the base case to find the answer for the end. ©Topperworld
- ✓ As we know, the base cases in the Fibonacci series are 0 and 1.
- ✓ Since the bottom approach starts from the base cases, so we will start from 0 and 1.

Let's understand through an example.

- ✓ Suppose we have an array that has 0 and 1 values at $a[0]$ and $a[1]$ positions, respectively shown as below:

0	1			
$a[0]$	$a[1]$			

- ✓ Since the bottom-up approach starts from the lower values, so the values at $a[0]$ and $a[1]$ are added to find the value of $a[2]$ shown as below:

0	1	1		
$a[0]$	$a[1]$	$a[2]$		

- ✓ The value of $a[3]$ will be calculated by adding $a[1]$ and $a[2]$, and it becomes 2 shown as below:

0	1	1	2	
$a[0]$	$a[1]$	$a[2]$	$a[3]$	

- ✓ The value of $a[4]$ will be calculated by adding $a[2]$ and $a[3]$, and it becomes 3 shown as below:

0	1	1	2	3	
$a[0]$	$a[1]$	$a[2]$	$a[3]$	$a[4]$	

- ✓ The value of $a[5]$ will be calculated by adding the values of $a[4]$ and $a[3]$, and it becomes 5 shown as below:

0	1	1	2	3	5	
$a[0]$	$a[1]$	$a[2]$	$a[3]$	$a[4]$	$a[5]$	

The **code** for implementing the Fibonacci series using the bottom-up approach is given below:

```

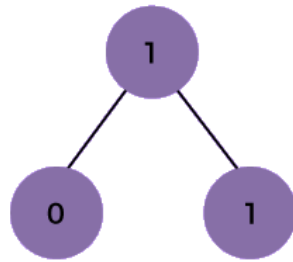
1. int fib(int n)
2. {
3.   int A[];
4.   A[0] = 0, A[1] = 1;
5.   for( i=2; i<=n; i++)
6.   {
7.     A[i] = A[i-1] + A[i-2]
8.   }
9.   return A[n];
10. }
```

Let's understand through the diagrammatic representation.

Initially, the first two values, i.e., 0 and 1 can be represented as:

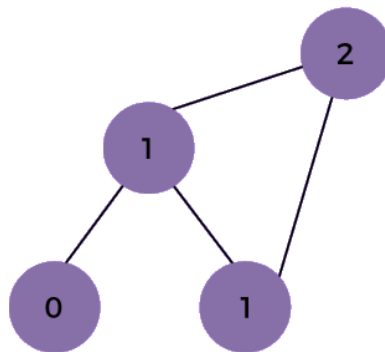


When $i=2$ then the values 0 and 1 are added shown as below:

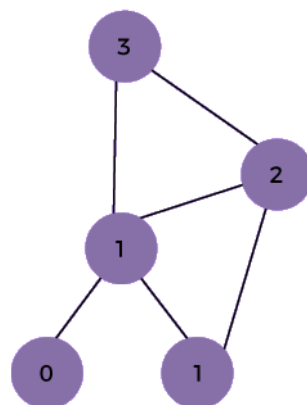


©Topperworld

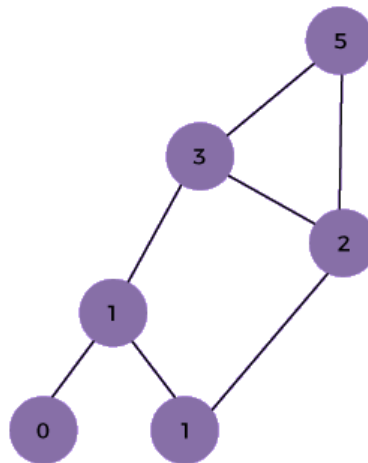
When $i=3$ then the values 1 and 1 are added shown as below:



When $i=4$ then the values 2 and 1 are added shown as below:



When $i=4$ then the values 2 and 1 are added shown as below:



In the above case, we are starting from the bottom and reaching to the top.

©Topperworld

Elements of Dynamic programming :

Dynamic programming is a powerful technique used in algorithm design to solve optimization problems by breaking them down into simpler subproblems and solving each subproblem only once, storing the solutions to subproblems in a table to avoid redundant computations.

Here are the key elements of dynamic programming in the context of designing algorithms:

➤ Optimal Substructure:

- ✓ Dynamic programming problems must have an optimal substructure, meaning that an optimal solution to the problem can be constructed from optimal solutions of its subproblems.

➤ Overlapping Subproblems:

- ✓ Dynamic programming problems should exhibit overlapping subproblems, meaning that the problem can be broken down into smaller, overlapping subproblems that are solved independently.
- ✓ The solutions to these subproblems are then stored for future use to avoid redundant computations.

➤ **Memoization or Tabulation:**

Dynamic programming solutions can be implemented using either **memoization or tabulation**.

Memoization:

- ✓ Involves storing the solutions to subproblems in a data structure (like a table or dictionary) and looking up the solution if the same subproblem is encountered again.

Tabulation:

- ✓ Involves building a table and systematically filling it out in a bottom-up manner, starting with the simplest subproblems and using their solutions to compute the solution to the larger problem.

Recurrence Relation:

- ✓ Dynamic programming problems are often defined by recurrence relations, which express the solution to a problem in terms of the solutions to its smaller subproblems.
- ✓ The recurrence relation provides a way to express the problem's solution recursively.

➤ **State Transition:**

- ✓ Dynamic programming problems involve defining how the state of the system changes as it moves from one solution to the next.
- ✓ Understanding how to transition from one state to another is crucial in formulating the recurrence relations and solving the problem.

➤ **Initialization:**

- ✓ Initialization of the base cases is essential to kickstart the dynamic programming process.
- ✓ Base cases are the simplest subproblems that can be solved directly without further decomposition.

➤ **Optimal Value:**

- ✓ The final solution to the dynamic programming problem is derived from the optimal values stored in the memoization table or computed during the tabulation process.

➤ **Time Complexity Analysis:**

- ✓ Analyzing the time complexity of a dynamic programming solution is crucial.
- ✓ Dynamic programming reduces time complexity by avoiding redundant computations, and understanding the efficiency of the algorithm is essential for practical implementation.

➤ **Space Complexity Analysis:**

- ✓ Dynamic programming solutions may have different space complexities depending on whether memoization or tabulation is used.
- ✓ Efficient use of memory is crucial, especially for large-scale problems.

❖ **LCS (Longest Common Subsequence)**

- LCS (Longest Common Subsequence) is a classic problem in dynamic programming and algorithm design.
- It is commonly studied in the field of Design and Analysis of Algorithms (DAA).
- The problem involves finding the longest subsequence that is common to two given sequences.
- A subsequence is a sequence that appears in the same order as the original sequence but not necessarily consecutively.

Problem Statement:

Given **two** sequences, say **sequence X** and **sequence Y**, the task is to find the length of the longest common subsequence of **both** sequences.

LCS Algorithm:

LCS-LENGTH (X, Y)

1. $m \leftarrow \text{length}[X]$
2. $n \leftarrow \text{length}[Y]$
3. for $i \leftarrow 1$ to m
4. do $c[i,0] \leftarrow 0$
5. for $j \leftarrow 0$ to n
6. do $c[0,j] \leftarrow 0$
7. for $i \leftarrow 1$ to m
8. do for $j \leftarrow 1$ to n
9. do if $x_i = y_j$
10. then $c[i,j] \leftarrow c[i-1,j-1] + 1$
11. $b[i,j] \leftarrow "\nwarrow"$
12. else if $c[i-1,j] \geq c[i,j-1]$
13. then $c[i,j] \leftarrow c[i-1,j]$
14. $b[i,j] \leftarrow "\uparrow"$
15. else $c[i,j] \leftarrow c[i,j-1]$
16. $b[i,j] \leftarrow "\leftarrow"$
17. return c and b .

©Topperworld

Longest Common Subsequence (LCS) using Bottom-Up (Tabulation):

We can use the following steps to implement the dynamic programming approach for LCS.

- Create a **2D array dp[][]** with rows and columns equal to the length of each input string **plus 1** [the number of rows indicates the indices of **S1** and the columns indicate the indices of **S2**].
- Initialize the first **row and column** of the dp array to **0**.
- Iterate through the rows of the dp array, starting from 1 (say using iterator i).
- For each i, iterate all the columns from **j = 1 to n**:
- If **S1[i-1]** is equal to **S2[j-1]**, set the current element of the dp array to the value of the element to **(dp[i-1][j-1] + 1)**.
- Else, set the current element of the dp array to the maximum value of **dp[i-1][j]** and **dp[i][j-1]**.
- After the nested loops, the last element of the dp array will contain the length of the **LCS**.

See the below illustration for a better understanding:

Say the strings are **S1 = "AGGTAB"** and **S2 = "GXTXAYB"**.

First step:

- ✓ Initially create a 2D matrix (say dp[][]) of size 8 x 7 whose first row and first column are filled with **0**.

		G	X	T	X	A	Y	B
	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
A	1	0						
G	2	0						
G	3	0						
T	4	0						
A	5	0						
B	6	0						

Creating dp table and filling 0 in 0th row and column

Second step:

- ✓ Traverse for $i = 1$. When j becomes 5, $S1[0]$ and $S2[4]$ are equal. So the $dp[i][j]$ is updated.
- ✓ For the other elements take the maximum of $dp[i-1][j]$ and $dp[i][j-1]$.
- ✓ (In this case, if both values are equal, we have used arrows to the previous rows).

©Topperworld

		G	X	T	X	A	Y	B	
		0	1	2	3	4	5	6	7
	0	0	0	0	0	0	0	0	0
A	1	0	0	0	0	0	1	1	1
G	2	0							
G	3	0							
T	4	0							
A	5	0							
B	6	0							

Updating dp table for row 1

Third step:

- ✓ While traversed for $i = 2$, $S1[1]$ and $S2[0]$ are the same (both are 'G').
- ✓ So the dp value in that cell is updated.
- ✓ Rest of the elements are updated as per the conditions.

		G	X	T	X	A	Y	B
	0	1	2	3	4	5	6	7
	0	0	0	0	0	0	0	0
A	1	0	0	0	0	1	1	1
G	2	0	1	1	1	1	1	1
G	3	0						
T	4	0						
A	5	0						
B	6	0						

Updating dp table for row 2

Fourth step:

- ✓ For $i = 3$, $S1[2]$ and $S2[0]$ are again same.

The updations are as follows.

		G	X	T	X	A	Y	B	
		0	1	2	3	4	5	6	7
	0	0	0	0	0	0	0	0	0
A	1	0	0	0	0	0	1	1	1
G	2	0	1	1	1	1	1	1	1
G	3	0	1	1	1	1	1	1	1
T	4	0							
A	5	0							
B	6	0							

Updating dp table for row 3

Fifth step:

- ✓ For $i = 4$, we can see that $S1[3]$ and $S2[2]$ are same.
- ✓ So $dp[4][3]$ updated as $dp[3][2] + 1 = 2$.

		G	X	T	X	A	Y	B	
	0	1	2	3	4	5	6	7	
	0	0	0	0	0	0	0	0	
A	1	0	0	0	0	0	1	1	1
G	2	0	1	1	1	1	1	1	1
G	3	0	1	1	1	1	1	1	1
T	4	0	1	1	2	2	2	2	2
A	5	0							
B	6	0							

Updating dp table for row 4

Sixth step:

- ✓ Here we can see that for $i = 5$ and $j = 5$ the values of $S1[4]$ and $S2[4]$ are same (i.e., both are 'A').
- ✓ So $dp[5][5]$ is updated accordingly and becomes 3.

		G	X	T	X	A	Y	B	
		0	1	2	3	4	5	6	7
	0	0	0	0	0	0	0	0	0
A	1	0	0	0	0	0	1	1	1
G	2	0	1	1	1	1	1	1	1
G	3	0	1	1	1	1	1	1	1
T	4	0	1	1	2	2	2	2	2
A	5	0	1	1	2	2	3	3	3
B	6	0							

Updating dp table for row 5

Final step:

- ✓ For $i = 6$, see the last characters of both strings are same (they are 'B'). Therefore the value of $dp[6][7]$ becomes 4.

		G	X	T	X	A	Y	B	
	0	1	2	3	4	5	6	7	
	0	0	0	0	0	0	0	0	
A	1	0	0	0	0	0	1	1	1
G	2	0	1	1	1	1	1	1	1
G	3	0	1	1	1	1	1	1	1
T	4	0	1	1	2	2	2	2	2
A	5	0	1	1	2	2	3	3	3
B	6	0	1	1	2	2	3	3	4

Updating dp table for row 6

So we get the maximum **length of common subsequence as 4.**

❖ MCM(Matrix Chain Multiplication)

- **Matrix Chain Multiplication (MCM)** is a classic problem in the field of Design and Analysis of Algorithms (DAA).
- In **MCM**, you are given a sequence of matrices, and the goal is to find the most efficient way to parenthesize these matrices to minimize the total number of scalar multiplications required to compute their product.

Algorithm of Matrix Chain Multiplication

MATRIX-CHAIN-ORDER (p)

```

1. n ← length[p]-1
2. for i ← 1 to n
3. do m[i, i] ← 0
4. for l ← 2 to n // l is the chain length
5. do for i ← 1 to n-l + 1
6. do j ← i + l - 1
7. m[i, j] ← ∞
8. for k ← i to j-1
9. do q ← m[i, k] + m[k + 1, j] + pi-1 pk pj
10. If q < m[i, j]
11. then m[i, j] ← q
12. s[i, j] ← k
13. return m and s

```

We will use table s to construct an optimal solution.

Step 1: Constructing an Optimal Solution:

```

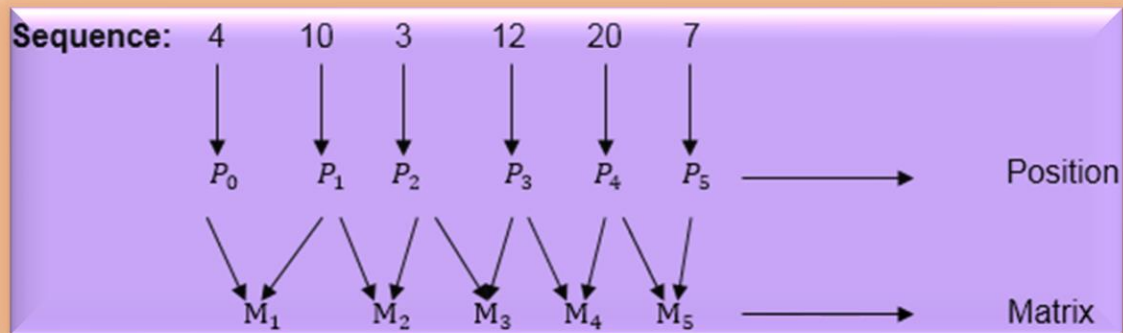
PRINT-OPTIMAL-PARENS (s, i, j)
1. if i=j
2. then print "A"
3. else print "("
4. PRINT-OPTIMAL-PARENS (s, i, s [i, j])
5. PRINT-OPTIMAL-PARENS (s, s [i, j] + 1, j)
6. print ")"

```

Example of Matrix Chain Multiplication**Example:**

- ✓ We are given the sequence {4, 10, 3, 12, 20, and 7}.
- ✓ The matrices have size 4 x 10, 10 x 3, 3 x 12, 12 x 20, 20 x 7.
- ✓ We need to compute $M[i, j]$, $0 \leq i, j \leq 5$. We know $M[i, i] = 0$ for all i .

	1	2	3	4	5	
1	0					1
2		0				2
3			0			3
4				0		4
5					0	5



Let us proceed with working away from the diagonal. We compute the optimal solution for the product of **2 matrices**.

Here P_0 to P_5 are Position and M_1 to M_5 are matrix of size $(p_i \text{ to } p_{i-1})$

On the basis of sequence, we make a formula

For $M_i \longrightarrow p[i]$ as column
 $p[i-1]$ as row

- ✓ In Dynamic Programming, initialization of every method done by '0'. So we initialize it by '0'. It will sort out diagonally.
- ✓ We have to sort out all the combination but the minimum output combination is taken into consideration.

Calculation of Product of **2 matrices**:

$$1. m(1,2) = m_1 \times m_2$$

$$= 4 \times 10 \times 10 \times 3$$

$$= 4 \times 10 \times 3 = 120$$

$$2. m(2,3) = m_2 \times m_3$$

$$= 10 \times 3 \times 3 \times 12$$

$$= 10 \times 3 \times 12 = 360$$

$$3. m(3, 4) = m_3 \times m_4$$

$$= 3 \times 12 \times 12 \times 20$$

$$= 3 \times 12 \times 20 = 720$$

$$4. m(4, 5) = m_4 \times m_5$$

$$= 12 \times 20 \times 20 \times 7$$

$$= 12 \times 20 \times 7 = 1680$$

1	2	3	4	5	
0	120				1
	0	360			2
		0	720		3
			0	1680	4
				0	5

- We initialize the diagonal element with equal i,j value with '0'.
- After that second diagonal is sorted out and we get all the values corresponded to it.

Now the third diagonal will be solved out in the same way.

Now product of **3 matrices**:

$$M[1, 3] = M_1 M_2 M_3$$

1. There are two cases by which we can solve this multiplication:
($M_1 \times M_2$) + M_3 , M_1 + ($M_2 \times M_3$)

2. After solving both cases we choose the case in which minimum output is there.

$$M[1, 3] = \min \begin{cases} M[1,2] + M[3,3] + p_0 p_2 p_3 = 120 + 0 + 4.3.12 = 264 \\ M[1,1] + M[2,3] + p_0 p_1 p_3 = 0 + 360 + 4.10.12 = 840 \end{cases}$$

M [1, 3] =264

As Comparing both output **264** is minimum in both cases so we insert **264** in table and **(M1 x M2) + M3** this combination is chosen for the output making.

M [2, 4] = M2 M3 M4

1. There are two cases by which we can solve this multiplication: **(M2x M3)+M4, M2+(M3 x M4)**
2. After solving both cases we choose the case in which minimum output is there.

$$M[2, 4] = \min \begin{cases} M[2,3] + M[4,4] + p_1 p_3 p_4 = 360 + 0 + 10.12.20 = 2760 \\ M[2,2] + M[3,4] + p_1 p_2 p_4 = 0 + 720 + 10.3.20 = 1320 \end{cases}$$

M [2, 4] = 1320

As Comparing both output **1320** is minimum in both cases so we insert **1320** in table and **M2+(M3 x M4)** this combination is chosen for the output making.

M [3, 5] = M3 M4 M5

1. There are two cases by which we can solve this multiplication: (**M3 x M4**) + **M5**, **M3+ (M4xM5)**
2. After solving both cases we choose the case in which minimum output is there.

$$M[3, 5] = \min \left\{ \begin{array}{l} M[3,4] + M[5,5] + p_2p_4p_5 = 720 + 0 + 3 \cdot 20 \cdot 7 = 1140 \\ M[3,3] + M[4,5] + p_2p_3p_5 = 0 + 1680 + 3 \cdot 12 \cdot 7 = 1932 \end{array} \right\}$$

M [3, 5] = 1140

As Comparing both output **1140** is minimum in both cases so we insert **1140** in table and (**M3 x M4**) + **M5** this combination is chosen for the output making.

1	2	3	4	5		1	2	3	4	5	
0	120				1	0	120	264			1
	0	360			2		0	360	1320		2
		0	720		3			0	720	1140	3
			0	1680	4				0	1680	4
				0	5					0	5

Now Product of 4 matrices:

M [1, 4] = M1 M2 M3 M4

There are three cases by which we can solve this multiplication:

1. (**M1 x M2 x M3**) **M4**
2. **M1 x(M2 x M3 x M4)**
3. (**M1 xM2**) x (**M3 x M4**)

After solving these cases we choose the case in which minimum output is there:

$$M[1, 4] = \min \begin{cases} M[1,3] + M[4,4] + p_0p_3p_4 = 264 + 0 + 4.12.20 = 1224 \\ M[1,2] + M[3,4] + p_0p_2p_4 = 120 + 720 + 4.3.20 = 1080 \\ M[1,1] + M[2,4] + p_0p_1p_4 = 0 + 1320 + 4.10.20 = 2120 \end{cases}$$

M [1, 4] =1080

As comparing the output of different cases then '**1080**' is minimum output, so we insert **1080** in the table and **(M1 xM2) x (M3 x M4)** combination is taken out in output making,

M [2, 5] = M2 M3 M4 M5

There are **three** cases by which we can solve this multiplication:

1. **(M2 x M3 x M4)x M5**
2. **M2 x(M3 x M4 x M5)**
3. **(M2 x M3)x (M4 x M5)**

After solving these cases we choose the case in which minimum output is there

$$M[2, 5] = \min \begin{cases} M[2,4] + M[5,5] + p_1p_4p_5 = 1320 + 0 + 10.20.7 = 2720 \\ M[2,3] + M[4,5] + p_1p_3p_5 = 360 + 1680 + 10.12.7 = 2880 \\ M[2,2] + M[3,5] + p_1p_2p_5 = 0 + 1140 + 10.3.7 = 1350 \end{cases}$$

M [2, 5] = 1350

As comparing the output of different cases then '**1350**' is minimum output, so we insert **1350** in the table and **M2 x (M3 x M4 xM5)** combination is taken out in output making.

1	2	3	4	5	1	2	3	4	5
0	120	264			0	120	264	1080	
	0	360	1320			0	360	1320	1350
		0	720	1140			0	720	1140
			0	1680				0	1680
				0					0

Now Product of 5 matrices:

M [1, 5] = M1 M2 M3 M4 M5

There are **five** cases by which we can solve this multiplication:

1. **(M1 x M2 xM3 x M4)x M5**
2. **M1 x(M2 xM3 x M4 xM5)**
3. **(M1 x M2 xM3)x M4 xM5**
4. **M1 x M2x(M3 x M4 xM5)**

After solving these cases we choose the case in which minimum output is there

$$M [1, 5] = \min \begin{cases} M[1,4] + M[5,5] + p_0 p_4 p_5 = 1080 + 0 + 4.20.7 = 1544 \\ M[1,3] + M[4,5] + p_0 p_3 p_5 = 264 + 1680 + 4.12.7 = 2016 \\ M[1,2] + M[3,5] + p_0 p_2 p_5 = 120 + 1140 + 4.3.7 = 1344 \\ M[1,1] + M[2,5] + p_0 p_1 p_5 = 0 + 1350 + 4.10.7 = 1630 \end{cases}$$

M [1, 5] = 1344

As comparing the output of different cases then '**1344**' is minimum output, so we insert **1344** in the table and **M1 x M2 x(M3 x M4 x M5)** combination is taken out in output making.

Final Output is:

1	2	3	4	5		1	2	3	4	5	
0	120	264	1080		1	0	120	264	1080	1344	1
	0	360	1320	1350	2		0	360	1320	1350	2
		0	720	1140	3			0	720	1140	3
			0	1680	4				0	1680	4
				0	5					0	5

Step 3:

Computing Optimal Costs:

- ✓ let us assume that matrix **A_i** has dimension **p_{i-1} x p_i** for **i=1, 2, 3.....n**.
- ✓ The input is a sequence **(p₀, p₁, p_n)** where length **[p] = n+1**.
- ✓ The procedure uses an auxiliary table **m [1.....n, 1.....n]** for storing **m [i, j]** costs an auxiliary table **s [1.....n, 1.....n]** that record which index of **k** achieved the optimal costs in computing **m [i, j]**.
- ✓ The algorithm first computes **m [i, j] ← 0** for **i=1, 2, 3.....n**, the minimum costs for the chain of length **1**.

❖ Time and Space Complexity

Time Complexity of MCM:

- ✓ The time complexity of the MCM algorithm using dynamic programming is **O(n³)**, where **n** is the number of matrices.

Space Complexity of MCM:

- ✓ The space complexity is **$O(n^2)$** to store the table of solutions to subproblems.

Time Complexity of LCS:

- ✓ The time complexity of the bottom-up dynamic programming solution for LCS is **$O(m * n)$**

Space Complexity of LCS:

- ✓ The space complexity of the bottom-up dynamic programming solution is **$O(m * n)$** as well **$(m * n)$** , where 'm' is the length of the first sequence and 'n' is the length of the second sequence.